Listing 6.13 Copying the `GetCityOptions` method and adding it to the `IndexModel`

```
private SelectList GetCityOptions()
{
    var cities = new List<City>
    {
        new City{ Id = 1, Name = "London"},
        new City{ Id = 2, Name = "Paris" },
        new City{ Id = 3, Name = "New York" },
        new City{ Id = 4, Name = "Rome" },
        new City{ Id = 5, Name = "Dublin" }
    };
    return new SelectList(cities, nameof(City.Id), nameof(City.Name));
}
```

Finally, assign the `Cities` in the `OnGet` method. Then add an `OnPost` method that works with the bound values in the `SelectedCities` property and uses them as a filter on the `Cities` collection to extract details of the selected cities.

Listing 6.14 Adding the handler methods to the `IndexModel` to set the city options

```
public void OnGet()
{
    Cities = GetCityOptions();
}
public void OnPost()
{
    Cities = GetCityOptions();
    if (ModelState.IsValid)
    {
        var cityIds = SelectedCities.Select(x => x.ToString());      ❶
        var cities = GetCityOptions().Where(o =>
cityIds.Contains(o.Value)).Select(o=>o.Text);                        ❷
        Message = $"You selected {string.Join(", ", cities)}";       ❸
    }
}
```

❶ Convert the collection of ints to strings.

❷ Filter the city options to select only those that have been selected.

❸ Use the string.Join method to construct the message from the resulting collection.